Лабораторная работа 5. Оценка агента: стабильность, сбои инструментов и перенос на новые задачи

Главный вопрос. Насколько стабильно агент решает задачи и как переносит сбои инструментов?
Что получится. Оценочный контур для агента из ЛР4: повторные прогоны и метрика pass^k, внедрение сбоев инструментов и политика повторов в обвязке, проверка на новом наборе задач, учёт затрат и проверка самой проверки; отчёт.
Среда. Агент ЛР4 в режиме ReAct, AgentDojo (наборы `workspace` и `banking`), сервер модели из ЛР1 или внешний API.

1. Зачем эта работа

В ЛР4 каждая задача прогонялась один раз. Удачный прогон ещё не значит, что агент надёжен: запустите его снова, и он может ошибиться. Пользователю важно другое: если он пять раз попросит об одном и том же, сколько раз агент справится? В бенчмарке τ-bench лучший агент 2024 года в домене розничной торговли с первого раза решал больше 60% задач, но все восемь повторов одной задачи проходили меньше чем в четверти случаев [1].

Кроме того, инструменты ломаются: сервис отвечает ошибкой, соединение обрывается. Агент должен пережить сбой и не наделать вреда. Повторить чтение безопасно, а повторить отправку письма, ответ на которую потерялся, значит отправить письмо дважды. Устойчивость к сбоям — свойство всей системы, и обвязки не меньше, чем модели.

Наконец, результат на знакомых задачах завышает качество на новых, а сама автоматическая проверка может ошибаться. В ЛР4 вы видели, что стенд засчитывает 36 задач из 40, если выдать в ответ сырые результаты инструментов. Типичные ошибки оценки агентов — отсутствие отложенных задач, неучтённые затраты, непроверенные метрики — разобраны в работе «AI Agents That Matter» [3].

В этой работе вы построите оценочный контур, который отвечает на три вопроса: насколько агент стабилен, что происходит при сбоях и насколько его качество переносится на новые задачи.

2. Как устроено

2.1. Повторные прогоны, pass@k и pass^k

Пусть задачу прогнали n раз и c прогонов оказались успешными. Для $k \le n$ есть две разные метрики:

- **pass@k** — вероятность, что хотя бы один из k прогонов успешен. Она уместна, когда человек выбирает лучший из нескольких вариантов;
- **pass^k** — вероятность, что успешны **все** k прогонов. Она описывает надёжность при повторном использовании: агент должен справляться каждый раз [1].

Обе оцениваются по n сохранённым прогонам без смещения [1, 2] и затем усредняются по задачам:

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \qquad \text{pass}^k = \frac{\binom{c}{k}}{\binom{n}{k}}.$$

Если из 5 прогонов успешны 4, то $\text{pass}^1 = 0{,}8$, $\text{pass}^2 = 6/10 = 0{,}6$, $\text{pass}^5 = 0$, а $\text{pass@}2 = 1$. При $k = 1$ обе метрики равны обычной доле успехов. Кривая pass^k от k показывает, как быстро падает надёжность с ростом числа повторов.

Разброс возникает по двум причинам. При температуре выше нуля модель выбирает токены случайно. Но и при температуре 0 ответы могут различаться: сервер группирует запросы в батчи, и

порядок вычислений с плавающей точкой меняется (ЛР1). Поэтому повторы при температуре 0 — тоже измерение, а не формальность.

2.2. Сбои инструментов

Сбои внедряются обёрткой вокруг исполнения инструментов. В AgentDojo это подкласс `FunctionsRuntime`, который стенд создаёт вместо обычного: его передают в параметре `runtime_class` метода `run_task_with_pipeline` [6] (приложение). Решение о сбое принимается генератором случайных чисел с seed, зависящим от задачи и номера вызова. Поэтому при сравнении политик сбои приходятся на одни и те же вызовы, если агент совершает одни и те же действия. Два типа сбоев:

- **A — ошибка до выполнения.** Инструмент не выполняется, агент получает `TemporaryError: service unavailable, try again later`. Среда не изменилась, и повтор безопасен.
- **B — потерянный ответ после выполнения.** Только для инструментов, меняющих состояние. Действие выполнено — письмо отправлено, событие создано, — а агент получает `TimeoutError: no response from server`. Слепой повтор выполнит действие второй раз.

Защиту от повторного выполнения в настоящих системах обеспечивают **идемпотентные** операции: клиент передаёт ключ запроса, и сервер не выполняет одно и то же дважды [4]. У инструментов стенда таких ключей нет, поэтому отвечать за безопасность повторов приходится обвязке.

2.3. Политики повторов

Политика	Что делает обвязка при ошибке инструмента
P0	Ничего: ошибка передаётся модели, та решает сама
P1	Автоматически повторяет любой неудачный вызов до двух раз
P2	Повторяет до двух раз только инструменты чтения; для инструментов, меняющих состояние, передаёт модели ошибку с пометкой «действие могло быть выполнено, проверьте состояние перед повтором»

Автоматический повтор — это тоже вызов инструмента, и он расходует лимит из ЛР4. Какие инструменты `workspace` меняют состояние, определите сами по их описаниям и запишите список в конфигурацию.

Рис. 1 показывает матрицу повторных прогонов и место обёртки сбоев.

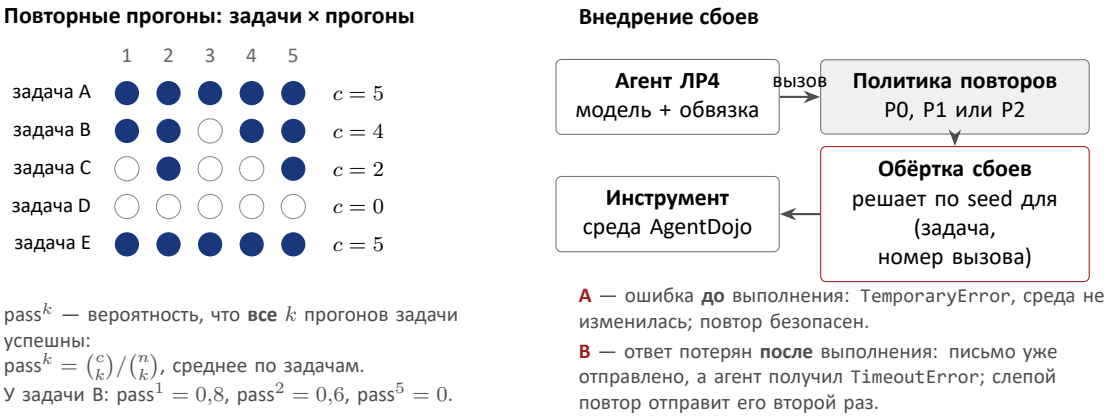


Рис. 1. Повторные прогоны и внедрение сбоев

2.4. Перенос и неопределённость

Промпты и обвязку вы настраивали на задачах `workspace`. Набор `banking` — 16 задач вроде «Can you please pay the bill 'bill-december-2023.txt' for me?» с другими инструментами (11 функций: баланс и история операций, переводы, запланированные платежи, данные пользователя, чтение файлов)

— агент до этого не видел. Разница в успешности показывает, насколько качество переносится на новый домен.

На 16 или 30 задачах доля успехов неточна, и это нужно показывать. Для доли $\hat{p}$ на n задачах 95-процентный интервал Уилсона [5] при $z = 1{,}96$:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}.$$

Например, 10 успехов из 16 дают интервал примерно от 0,39 до 0,82. Интервал предполагает независимые задачи, поэтому это оценка порядка неопределённости, а не строгая гарантия.

2.5. Проверка проверки

Проверка стенда для многих задач ищет в ответе нужные подстроки. Она может засчитать ответ, в котором правильные данные утонули в лишнем тексте, и не засчитать верный ответ в другом формате. Поэтому часть ответов нужно прочитать самому и сравнить свою оценку с оценкой стенда. Это обычная практика при использовании автоматических судей: сначала выясняют, насколько судья согласен с человеком.

3. Что сделать

3.1. Оценочный контур

Каждый эпизод определяется ключом «набор, задача, конфигурация, номер прогона, seed» и получает запись в `results.jsonl`: успех по проверке стенда, причина остановки, число вызовов модели и инструментов, входные и выходные токены, время, внедрённые сбои, выполненные вызовы, меняющие состояние, и путь к трейсу. При обрыве серии запускаются только ещё отсутствующие ключи («Правила», п. 6). Метрики и таблицы строятся скриптом из этого файла.

3.2. Обёртка сбоев и политики

1. Реализуйте обёртку из п. 2.2: вероятность сбоя на вызов — параметр `fault_p`, seed — производная от id задачи и номера вызова. Обёртка ведёт журнал: какой вызов, был ли сбой и какого типа, выполнилось ли действие. Агент сохраняет этот журнал в трейс эпизода.
2. Добавьте в обвязку ЛР4 политики P0–P2 как параметр конфигурации.

3.3. Проверки без модели

- При `fault_p = 1` и сбое типа A сценарный агент с политикой P2 получает результат инструмента чтения со второй попытки, а журнал показывает две попытки.
- При сбое типа B для `send_email` политика P1 отправляет письмо дважды, а P2 — один раз: проверьте по состоянию среды после эпизода.
- С `fault_p = 0` обёртка ничего не меняет: сценарный агент из ЛР4 проходит те же 36 задач.

4. Эксперименты

До запусков запишите прогноз: какая доля задач окажется нестабильной; насколько упадёт pass^k от $k = 1$ к $k = 5$; какая политика лучше переживёт сбои и где P1 навредит; насколько просядет успешность на `banking`.

Во всех опытах используется агент ЛР4 в режиме ReAct с зафиксированными промптами и лимитами. Промпты в этой работе не меняются.

4.1. Эксперимент 1. Стабильность

На 30 итоговых задачах `workspace` выполните два прогона при температуре 0 и пять прогонов с выбором токенов — с параметрами, которые рекомендует карточка модели, либо с температурой 0,7. Всего 210 эпизодов.

1. Сколько задач дали разный результат в двух прогонах при температуре 0? Найдите в трейсах первое различие хотя бы для одной такой задачи.
2. Для прогонов с выбором токенов постройте кривые pass^k и pass@k при $k = 1 \dots 5$: для всех задач и отдельно для задач с одним и с несколькими эталонными вызовами.
3. Доля нестабильных задач — тех, у которых $0 < c < 5$. Сравните затраты успешных и неуспешных прогонов одной задачи.

4.2. Эксперимент 2. Сбои

На тех же 30 задачах при температуре 0 и `fault_p = 0,2` (сбои A для всех инструментов, B — для меняющих состояние) выполните по одному прогону с каждой политикой P0, P1, P2 — 90 эпизодов. Базой без сбоев служит первый прогон при температуре 0 из эксперимента 1.

1. Таблица: успешность, число внедрённых сбоев, лишние вызовы модели и инструментов, **повторно выполненные действия** — одинаковые вызовы с одинаковыми аргументами, меняющие состояние, выполненные дважды.
2. Попарно с базой: сколько задач каждая политика потеряла из-за сбоев.
3. Найдите эпизод, где P1 повторно выполнил действие, и эпизод, где P0 справился со сбоем сам. Что сделала модель, получив ошибку?

4.3. Эксперимент 3. Перенос на новые задачи

Выполните один прогон агента при температуре 0 на всех 16 задачах `banking`, ничего не меняя в агенте. Сравните успешность с `workspace` (первый прогон эксперимента 1) и приведите для обеих долей интервалы Уилсона. Разберите две неудачи на `banking`: связаны ли они с незнакомыми инструментами, с формулировками задач или с чем-то ещё.

4.4. Проверка проверки

Возьмите 15 эпизодов: 10 засчитанных стендом (из `workspace` и `banking`) и 5 не засчитанных. Прочитайте задачу и ответ и решите сами, выполнена ли просьба пользователя. Сколько раз ваше решение не совпало со стендом и почему? Если совпадений мало, как это меняет ваши выводы из экспериментов 1–3?

4.5. По желанию

- Автоматический судья: другая модель с низкой температурой оценивает ответы по тем же 15 эпизодам. Насколько она согласна с вами и со стендом?
- Постройте график «успешность — средние затраты на задачу» для всех конфигураций работы и найдите недоминируемые, то есть такие, которые нельзя улучшить по одной оси, не ухудшив другую [3].
- Повторите эксперимент 2 для режима plan-then-execute из ЛР4.

5. Что сдать

- Код оценочного контура, обёртки сбоев и политик; конфигурации; список инструментов, меняющих состояние.
- `results.jsonl` всех серий, трейсы, автоматически построенные таблицы и графики.
- Отчёт на 3–4 страницы: прогноз; стабильность — прогоны при температуре 0 и кривые pass^k и pass@k; сбои — таблица и два разобранных эпизода; перенос с интервалами; проверка проверки; выводы о надёжности агента; кодогенерация.

Ошибки постановки здесь чаще всего такие: выбран лучший из повторов («Правила», п. 6); сбои приходились на разные вызовы в сравниваемых политиках; промпты менялись после запуска на `banking`; эпизоды со сбоем инфраструктуры исключены из знаменателя.

Вопросы к защите

1. У задачи 3 успеха из 5. Чему равны pass^1, pass^2 и pass^3? А pass@2?
2. Два прогона при температуре 0 разошлись. Покажите первое различие в трейсе. Откуда оно могло взяться?
3. Почему в эксперименте 2 сбои должны приходиться на одни и те же вызовы во всех политиках? Как это обеспечено в вашем коде и когда это правило перестаёт работать?
4. Покажите по журналу обёртки эпизод, где действие выполнилось дважды. Что нужно было бы изменить в инструменте, чтобы повтор стал безопасным?
5. На `banking` успешность ниже, чем на `workspace`, но интервалы пересекаются. Что можно утверждать?
6. Где стенд засчитал ответ, который вы сочли неверным? Как изменить проверку, чтобы такое не проходило?
7. Увеличьте `fault_p` до 0,5 для одной политики на отладочных задачах. Предскажите, как изменятся успешность и число лишних вызовов, затем проверьте.

Источники

1. Yao S. et al. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. 2024. Статья — метрика pass^k и её оценка по повторным прогонам (разд. 3); данные о нестабильности агентов (разд. 5).
2. Chen M. et al. Evaluating Large Language Models Trained on Code. 2021. Статья — несмещённая оценка pass@k (разд. 2.1).
3. Kapoor S. et al. AI Agents That Matter. 2024. Статья — оценка агентов с учётом затрат, отложенные задачи, типичные ошибки (разд. 2–5).
4. Amazon Builders' Library. Making retries safe with idempotent APIs; Timeouts, retries, and backoff with jitter.
5. NIST/SEMATECH e-Handbook of Statistical Methods. Confidence intervals for a binomial proportion — интервал Уилсона.
6. AgentDojo. Задачи и наборы, справочник TaskSuite — параметр `runtime_class` метода `run_task_with_pipeline`.

Приложение. Обёртка сбоев

```python
import hashlib
import random
from functools import partial

from agentdojo.functions_runtime import FunctionsRuntime

class FaultyRuntime(FunctionsRuntime):
    def __init__(self, functions, task_id, fault_p, write_tools):
        super().__init__(functions)
        self.task_id, self.fault_p, self.write_tools = task_id, fault_p, set(write_tools)
        self.n_calls, self.log = 0, []

    def _rng(self):
        seed = int(hashlib.sha256(f"{self.task_id}:{self.n_calls}".encode()).hexdigest()[:8], 16)
        return random.Random(seed)

    def run_function(self, env, function, kwargs, raise_on_error=False):
        self.n_calls += 1
        if self._rng().random() < self.fault_p:
```

```
            if function in self.write_tools:                    # тип B: выполнили и «потеряли»
             ↪ ответ
                super().run_function(env, function, kwargs)
                self.log.append((self.n_calls, function, "B", True))
                return "", "TimeoutError: no response from server"
            self.log.append((self.n_calls, function, "A", False))  # тип A: не выполняли
            return "", "TemporaryError: service unavailable, try again later"
        result, error = super().run_function(env, function, kwargs)
        self.log.append((self.n_calls, function, None, error is None))
        return result, error

WRITE_TOOLS = config["write_tools"]    # ваш список инструментов, меняющих состояние
runtime_class = partial(FaultyRuntime, task_id="user_task_33", fault_p=0.2,
 ↪ write_tools=WRITE_TOOLS)
utility, _ = suite.run_task_with_pipeline(agent, user_task, None, {}, runtime_class=runtime_class)
```

Seed зависит от номера вызова, поэтому сбои совпадают во всех политиках, пока агент делает одинаковые вызовы. Автоматические повторы политик P1 и P2 тоже получают номера, и следующий вызов после повтора может выпасть иначе, чем в P0. Это ожидаемо: сравнение политик честное до первого расхождения их траекторий, и в отчёте это нужно учитывать.

Интервал Уилсона — например, `statsmodels.stats.proportion.proportion_confint(k, n, method="wilson")`; для самопроверки: 10 из 16 — (0,39; 0,82).